

Object Oriented Programming Lab 03

Course: Object Oriented Programming (CL1004)

Semester: Spring 2026

Instructor: Izzah Salam

Note:

- Maintain discipline during the lab.
- Listen and follow the instructions as they are given.
- Just raise hand if you have any problem.
- Completing all tasks of each lab is compulsory.
- Get your lab checked at the end of the session.

Fundamentals of Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. It provides a structured approach to software development by organizing code into reusable and manageable components. OOP aims to implement real-world entities such as inheritance, polymorphism, encapsulation, and abstraction in programming, making it more intuitive, scalable, and efficient.

In C++, OOP allows programmers to design programs using classes as blueprints for creating objects, which encapsulate both data (attributes) and functions (methods) that operate on that data. This binding of data and functions into a single unit ensures data security and integrity, preventing unintended interference from other parts of the program.

Classes in C++

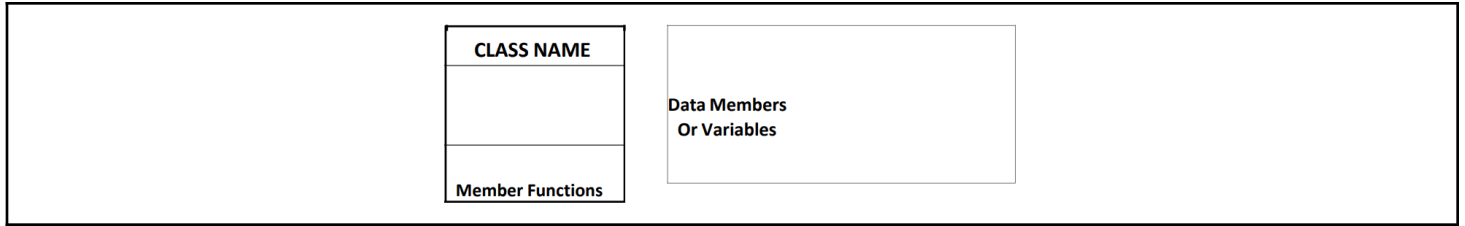
A class is a programmer-defined data type that describes what an object of the class will look like when it is created. It consists of a set of variables and a set of functions. We can think of class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows etc. based on these descriptions we build the house. House is the object.

As, many houses can be made from the same description, we can create many objects from a class.

Classes are created using the keyword class. A class declaration defines a new type that links code and data. This new type is then used to declare objects of that class. A Class is a user defined data-type which has data members and member functions. Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.

A class member can be defined as public, private or protected. By default, members would be assumed as private.

In the UML, a class icon can be subdivided into compartments. The top compartment is for the name of the class, the second is for the variables of the class, and the third is for the methods of the class.



Class Name

By convention, the name of a user-defined class begins with a capital letter and, for readability, each subsequent word in the class name begins with a capital letter.

Data Members

Consider the attributes of some real-world objects:

RADIO – station frequency, volume level.

CAR – speedometer readings, amount of gas in its tank and what gear it is in.

These attributes form the data in our program. The values that these attributes take (the blue color of the petals, for example) form the state of the object.

Member Functions

Consider the operations of some real-world objects:

RADIO – setting its station and volume (invoked by the person adjusting the radio's controls)

CAR – accelerating (invoked by the driver), decelerating, turning and shifting gears. These operations form the functions in program. Member functions define the class's behaviors.

Objects in C++

In C++, when we define a variable of a class, we call it instantiating the class. The variable itself is called an instance of the class. A variable of a class type is also called an object. Instantiating a variable allocates memory for the object. Below is the syntax to define an object in C++:

className objectVariableName

Following is an example of an object created from a class named as radio:

Radio r;

Accessing Public Data Members

The public data members of objects of a class can be accessed using the direct member access operator (.). However, the private data members are not allowed to be accessed directly by the object. Accessing a data member depends solely on the access control of that data member.

Accessing Private Data Members

To access, use and initialize the private data member you need to create getter and setter functions, to get and set the value of the data member. The setter function will set the value passed as argument to the private data member, and the getter function will return the value of the private data member to be used. Both getter and setter function must be defined public.

```
C++> oop.cpp> main()
1  #include<iostream>
2  #include <iomanip>
3  using namespace std;
4
5  class Student
6  {
7      private:    // private data member
8      int rollno;
9
10     public:
11     // public function to get value of rollno - getter
12     int getRollno()
13     {
14         return rollno;
15     }
16     // public function to set value for rollno - setter
17     void setRollno(int i)
18     {
19         rollno=i;
20     }
21 };
22
23 int main()
24 {
25     Student A;
26     A.rollno=1; //Compile time error
27     cout<< A.rollno; //Compile time error
28
29     A.setRollno(1); //Rollno initialized to 1
30     cout<< A.getRollno(); //Output will be 1
31 }
32
```

Setters

Setters are special functions in a class that change the values of private variables. They help keep the data safe by controlling how it's updated, preventing direct access to sensitive information. Using setters makes the code more secure and organized, ensuring that only valid data is stored. They also allow adding conditions or checks before assigning values, reducing errors. Setters are an important part of encapsulation in Object-Oriented Programming (OOP).

Getters

Getters are special functions in a class that help access private variables. They allow controlled retrieval of data without exposing the actual variable. Using getters makes the code more secure and keeps the data safe from unwanted changes.

Consider another example below:

```

#include<iostream>
using namespace std;
class Shape
{
    // as private so object.height and object.width is inaccessible
    int height, width;

public:
    //setters
    void setHeight(int h){
        height = h;
    }
    void setWidth(int w){
        width = w;
    }
    //getters
    int getHeight(){
        return height;
    }
    int getWidth(){
        return width;
    }
};

int main()
{
    Shape shape;

    // setters
    shape.setHeight(5);
    shape.setWidth(2);
    // getters
    cout << "The Height is : " << shape.getHeight() << endl;
    cout << "The Width is : " << shape.getWidth() << endl;

    // shape.height or shape.width wont work as they are private
    return 0;
}

```

Member Functions in Classes

There are 2 ways to define a member function:

1. Inside class definition.
2. Outside class definition.

Inside Class Definition

With an inline function, the compiler tries to expand the code in the body of the function in place of a call to the function. Note that all the member functions defined inside the class definition are by default inline, but you can also make any non-class function inline by using keyword inline with them. Inline functions are actual functions, which are copied everywhere during compilation, like pre-processor macro, so the overhead of function calling is reduced. This type of definition is already shown in the above examples.

Outside Class Definition

To define a member function outside the class definition we have to use the scope resolution:: operator along with class name and function name.

```

#include <iostream>
using namespace std;
class car
{
private:
int car_number;
char car_model[10];
public:
void getdata(); //function declaration
void showdata();
};
//Outside class function definition
void car::getdata()
{
cout<<"Enter car number: "; cin>>car_number;
cout<<"\n Enter car model: "; cin>>car_model;
}
//Outside class function definition
void car::showdata()
{
cout<<"Car number is "<<car_number;
cout<<"\n Car model is "<<car_model;
}
// main function starts
int main()
{
car c1;
c1.getdata();
c1.showdata();
return 0;
}

```

Types of Classes

- Concrete Class
- Generalized Class
- Specialized Class

Concrete Class

A concrete class is a class that provides an implementation for all of its methods. It cannot have any unimplemented methods.

```

#include<iostream>
#include<string>
using namespace std;

class Concrete {
private:
string info;
public:
Concrete(string s) : info(s) { }
void printContent() {
cout << "Concrete Object Information\n" << info << endl;
}
};

int main()
{
string s;
s = "Object Creation";
Concrete c(s);
c.printContent();
}

```

Generalized Class

A class that defines the main features but not the specific details is considered a general class. Classes situated at the top of the inheritance hierarchy are referred to as general classes.

```

#include <iostream>
#include<string>
using namespace std;
class Car {
public:
int price;
int year;
string make;
string model;
Car(int p, int y, string m, string mo) {
price = p;
year = y;
make = m;
model = mo;
}
void displayInformation() {
cout << "Price: " << price << endl;
cout << "Year: " << year << endl;
cout << "Make: " << make << endl;
cout << "Model: " << model << endl;
}
};

```

Specialized Class

A class that is highly specific and defines detailed characteristics is considered a specific class. Classes situated at the bottom of the inheritance hierarchy are referred to as specific classes.

In the code provided below, the class "ToyotaCars" is an example of a specialized class. This class represents a specific type of car, namely Toyota cars, and provides specific information about them, such as the model, year, and color. The class is defined with private variables to store this information, and public methods to display it.

```

#include<iostream>
#include<string>
using namespace std;
class ToyotaCars {
private:
string model;
int year;
string color;
public:
ToyotaCars(string model, int year, string color)
{
model = model;
year = year;
color = color;
}
void displayInfo()
{
cout<<"Model: "<<model<<endl;
cout<<"Year: "<<year<<endl;
cout<<"Color: "<<color<<endl;
}
};

```

Struct VS Classes

By default, all structure fields are public, or available to functions (like the main() function) that are outside the structure. Conversely, all class fields are private. That means they are not available for use outside the class. When you create a class, you can declare some fields to be private and some to be public. For example, in the real world, you might want your name to be public knowledge but your Social Security number, salary, or age to be private.

Transformation from Procedural to Object Oriented Programming

Procedural Approach

```

#include<iostream>
using namespace std;

double calculateBMI(double w, double h)
{
return w/(h*h)*703;
}

string findStatus(double bmi)
{
string status;
if(bmi < 18.5)
status = "underweight";
else if(bmi < 25.0)
status = "normal"; // so on.
return status;
}

int main()
{
double bmi, weight, height;
string status;
cout<<"Enter weight in Pounds ";
cin>>weight;
cout<<"Enter height in Inches ";
cin>>height;
bmi=calculateBMI(weight,height);
cout<<"Your BMI is "<<bmi<<" Your status is "<<findStatus(bmi);
}

```

Object Oriented Programming Approach

```

#include<iostream>
using namespace std;
class Account
{
private:
    double balance; // Account balance
public: //Public interface:
    string name; // Account holder long accountNumber;
    // Account number void setDetails(double bal)
    {
        balance = bal;
    }
    double getDetails()
    {
        return balance;
    }
    void displayDetails()
    {
        cout<<"Details are: "<<endl;
        cout<<"Account Holder:
        "<<name<<endl;
        cout<<"Account Number:
        "<<
        accountNumber <<endl; cout<<"Account
        Balance: "<<getDetails()<<endl;
    }
};

int main()
{ double accBal;
  Account
  currentAccount;
  currentAccount.getDetails();

  cout<<"Please enter the details"<<endl;
  cout<<"Enter Name:"<<endl; getline(cin,
  currentAccount.name); cout<<"Enter
  Account Number:"<<endl;
  cin>>currentAccount.accountNumber;

  cout<<"Enter Account
  Balance:"<<endl; cin>>accBal;
  currentAccount.setDetails(accBal);

  cout<<endl;
  currentAccount.displayDetails();
  return 0;
}

```

Set and get functions to manipulate
Private data member

Publically available data:
Assigning values from

Private data:

Accessing private data using member function

Header Files

Header files are used for declaration. In OOP you should use header files to declare classes and functions. It will make your program look cleaner and more professional.

Header file for a class will include:

- Include guards.
- Class definition:
 - Member variables.
 - Function declaration (only prototype).

Implementation File will include:

- Include directive for “header.h”.
- Necessary include directives.
- Function definitions for all the functions of the class.

Consider the below example:


```
// my_class.h
#ifndef MY_CLASS_H // include guard
#define MY_CLASS_H
```

```
namespace N
{
    class my_class
    {
        public:
        void do_something();
    };
}
```

```
// my_program.cpp #include
"my_class.cpp"
```

```
using namespace N;
```

```
int main()
{
    my_class mc; mc.do_something();
    return 0;
}
```

```
// my_class.cpp
#include "my_class.h" // header in local directory
#include <iostream> // header in standard library
```

```
using namespace N; using
namespace std;
```

```
void my_class::do_something()
{
    cout << "Doing something!" << endl;
}
```